

PART 1: Key Identification Exercises

Task 1.1 : Superkey and Candidate Key Analysis

Relation A: Employee

1) 1. (EmpID) 2. (SSN) 3. (Email) 4. (Phone) 5. (EmpID, Name) 6. (EmpID, Department)

2) 1. (EmpID) 2. (SSN) 3. (Email) 4. (Phone)

3) I would choose EmpID because it is short and unique.

4) Based solely on this sample data, one might assume that two employees cannot have the same phone number. However, database rules do not strictly mandate that the phone number field be unique. For example, family members living in the same household might list a shared work phone number.

Relation B: Course Registration

1) Primary Key: {StudentID, CourseCode, Section, Semester, Year}

2) Each attribute is necessary because StudentID identifies the student, CourseCode identifies the course, Section identifies the specific course section, and Semester and Year identify when the student registered. A student can take the same course in different semesters, so Semester and Year are required. Section is also required because a course can have different sections in the same semester.

3) Additional Candidate Keys: No additional candidate keys are specified by the given business rules.

Task 1.2: Foreign Key Design

1. Student.AdvisorID → Professor.ProfID

This relationship connects each student with their professor advisor.

2. Course.DepartmentCode → Department.DeptCode

This relationship identifies the department to which a course belongs.

3. Department.ChairID → Professor.ProfID

This relationship identifies the professor who is the chair of a department.

4. Enrollment.StudentID → Student.StudentID

This relationship connects an enrollment record with the student who is enrolled.

5. Enrollment.CourseID → Course.CourseID

This relationship connects an enrollment record with the course being taken.

PART 2: ER Diagram Construction

Task 2.1: Hospital Management System

1) Strong entities: Patient, Doctor, Department, Appointment, Prescription.

Weak entity: Room.

2) Patient: PatientID — Simple, Primary Key, Name — Simple, Birthdate — Simple, Address — Composite: Street, City, State, ZIP, Phone — Multivalued, Insurance — Simple.

Doctor: DoctorID — Simple, Primary Key, Name — Simple, Specialization — Multivalued, Phone — Simple, OfficeLocation — Simple.

Department: DeptCode — Simple, Primary Key, DeptName — Simple, Location — Simple.

Appointment: AppointmentID — Simple, Primary Key, DateTime — Simple, Purpose — Simple, Notes — Simple.

Prescription: PrescriptionID — Simple, Primary Key, Medication — Simple, Dosage — Simple, Instructions — Simple.

Room: RoomNumber — Simple, Partial Key.

3) Relationships:

Patient — Appointment: 1:N

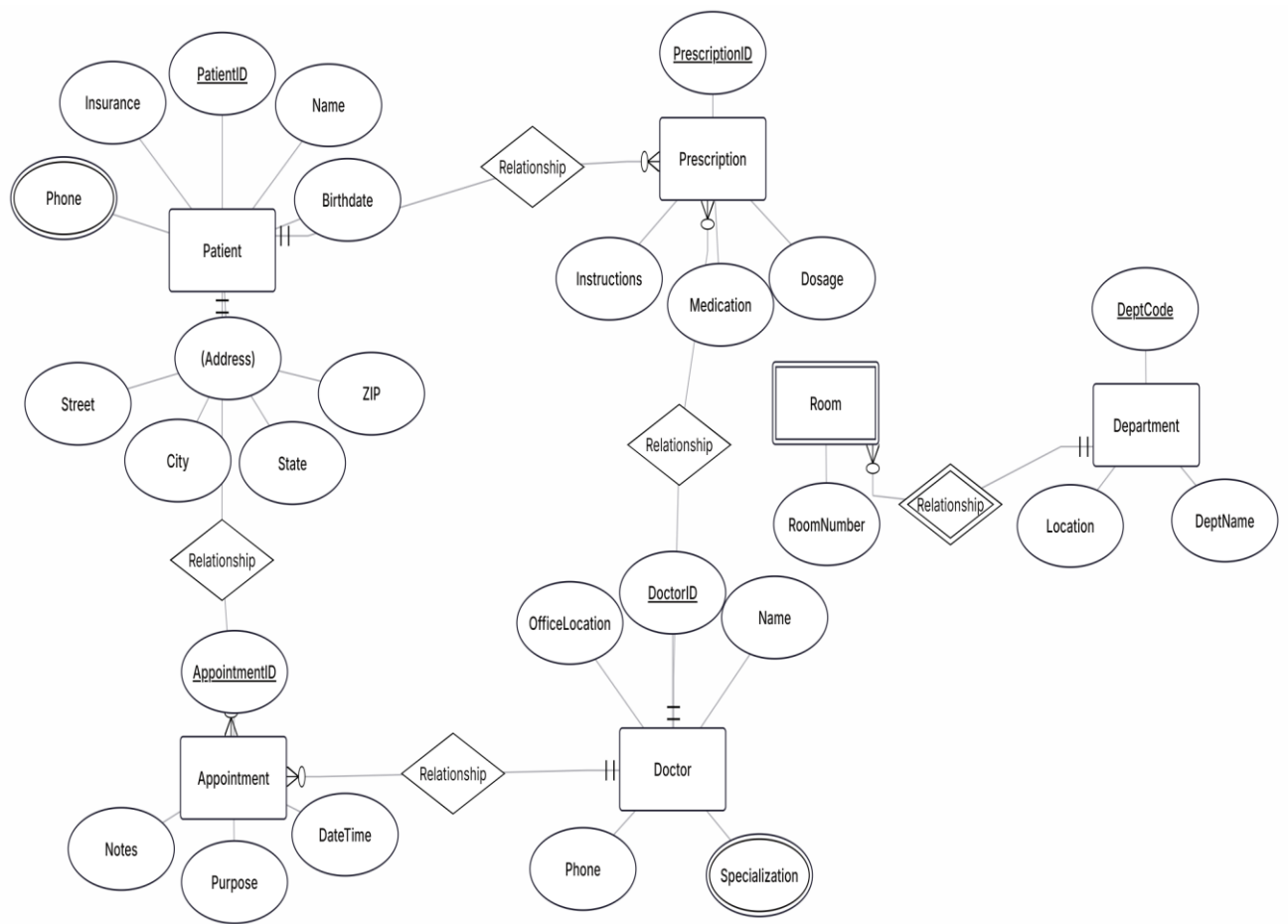
Doctor — Appointment: 1:N

Patient — Prescription: 1:N

Doctor — Prescription: 1:N

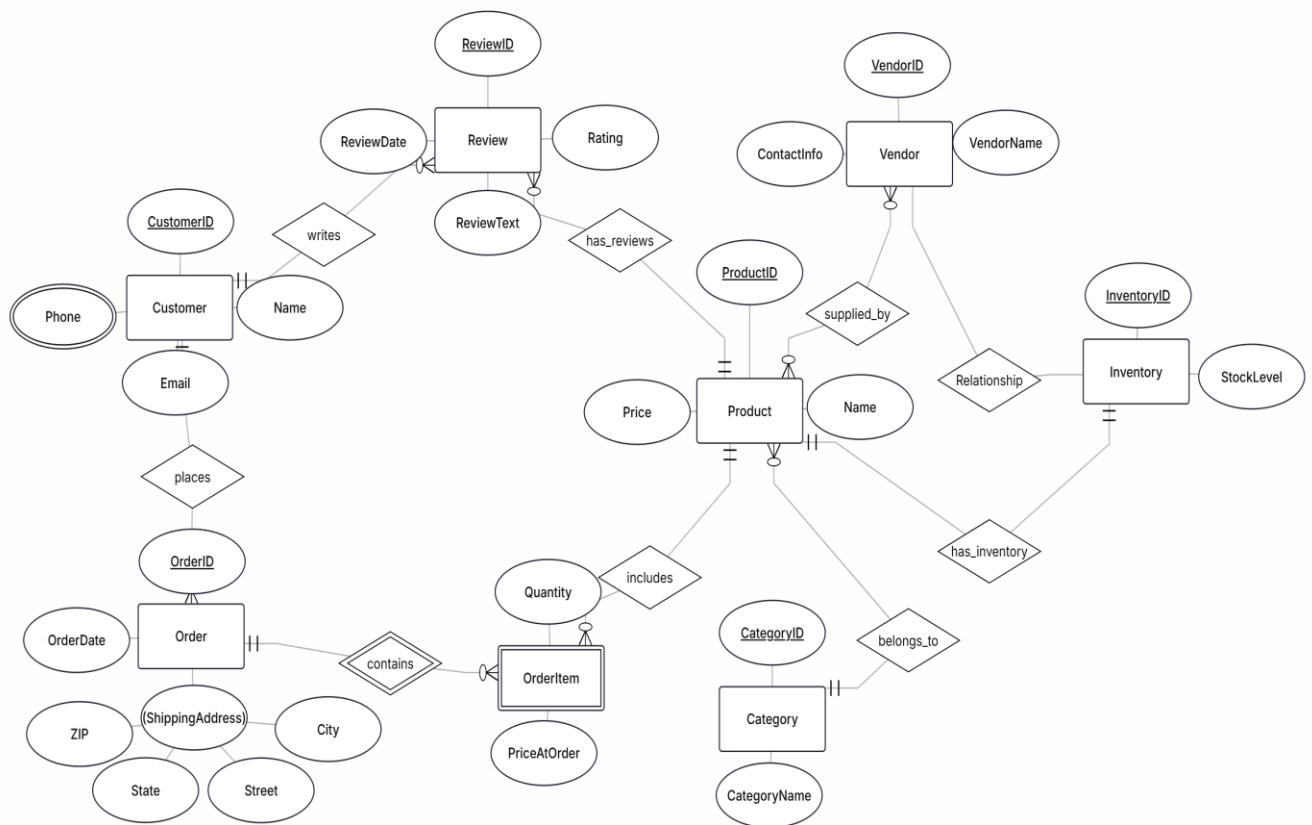
Department — Room: 1:N, Identifying Relationship

4/5)



Task 2.2: E-commerce Platform

1)



2) Weak Entity : OrderItem

Because an OrderItem is not fully defined in isolation, it exists as part of a specific Order. It has Quantity and PriceAtOrder attributes, and the relationship between Order and OrderItem is an identifying one.

3) Many-to-many relationship: Order — Product
Attributes: Quantity, PriceAtOrder

PART 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

- 1) $\text{StudentID} \rightarrow \text{StudentName}$
- 2) $\text{StudentID} \rightarrow \text{StudentMajor}$
- 3) $\text{ProjectID} \rightarrow \text{ProjectTitle}$
- 4) $\text{ProjectID} \rightarrow \text{ProjectType}$
- 5) $\text{ProjectID} \rightarrow \text{SupervisorID}$
- 6) $\text{SupervisorID} \rightarrow \text{SupervisorName}$
- 7) $\text{SupervisorID} \rightarrow \text{SupervisorDept}$
- 8) $(\text{StudentID}, \text{ProjectID}) \rightarrow \text{Role}$

9.(StudentID, ProjectID) → HoursWorked

10.(StudentID, ProjectID) → StartDate

11.(StudentID, ProjectID) → EndDate

2) 1. Update anomaly : If a supervisor's SupervisorDept changes, it must be updated across multiple rows. If one row is overlooked, the data becomes inconsistent.

2. Insert anomaly : It is difficult to fully enter information about a new project or supervisor until a student is involved, because the table depends on the student-project relationship.

3. Delete anomaly : If we delete the record of the last student participating in a project, information about that project and its supervisor might also be lost.

3) Apply 1NF:

There are no 1NF violations. The table is already in 1NF because all attributes contain atomic values.

Primary Key: (StudentID, ProjectID) — composite primary key.

4) Apply 2NF:

The table is not in 2NF because there are partial dependencies on the composite primary key (StudentID, ProjectID).

Decomposition:

1.Student(StudentID, StudentName, StudentMajor)

2.Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)

3.StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5) Apply 3NF:

The Project table has a transitive dependency because ProjectID → SupervisorID and SupervisorID → SupervisorName, SupervisorDept.

Final decomposition:

1.Student(StudentID, StudentName, StudentMajor)

2.Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)

3.Supervisor(SupervisorID, SupervisorName, SupervisorDept)

4.StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

Task 4.2: Advanced Normalization

1) Primary Key:

(StudentID, CourseID, TimeSlot) is the composite primary key because a

student can be enrolled in multiple course sections, and each course section is identified by its course and time slot.

2) Functional Dependencies:

1. $\text{StudentID} \rightarrow \text{StudentMajor}$
2. $\text{CourseID} \rightarrow \text{CourseName}$
3. $\text{InstructorID} \rightarrow \text{InstructorName}$
4. $(\text{TimeSlot}, \text{Room}) \rightarrow \text{Building}$
5. $(\text{CourseID}, \text{TimeSlot}) \rightarrow \text{InstructorID}$
6. $(\text{CourseID}, \text{TimeSlot}) \rightarrow \text{Room}$

3) The table is not in BCNF because several functional dependencies have determinants that are not superkeys.

Violations:

1. $\text{StudentID} \rightarrow \text{StudentMajor}$
2. $\text{CourseID} \rightarrow \text{CourseName}$
3. $\text{InstructorID} \rightarrow \text{InstructorName}$
4. $(\text{TimeSlot}, \text{Room}) \rightarrow \text{Building}$
5. $(\text{CourseID}, \text{TimeSlot}) \rightarrow \text{InstructorID}, \text{Room}$

Therefore, CourseSchedule is not in BCNF.

4) The CourseSchedule table is decomposed into the following BCNF relations:

1. Student(StudentID, StudentMajor)
2. Course(CourseID, CourseName)
3. Instructor(InstructorID, InstructorName)
4. CourseSection(CourseID, TimeSlot, InstructorID, Room)
5. RoomInfo(TimeSlot, Room, Building)
6. Enrollment(StudentID, CourseID, TimeSlot)

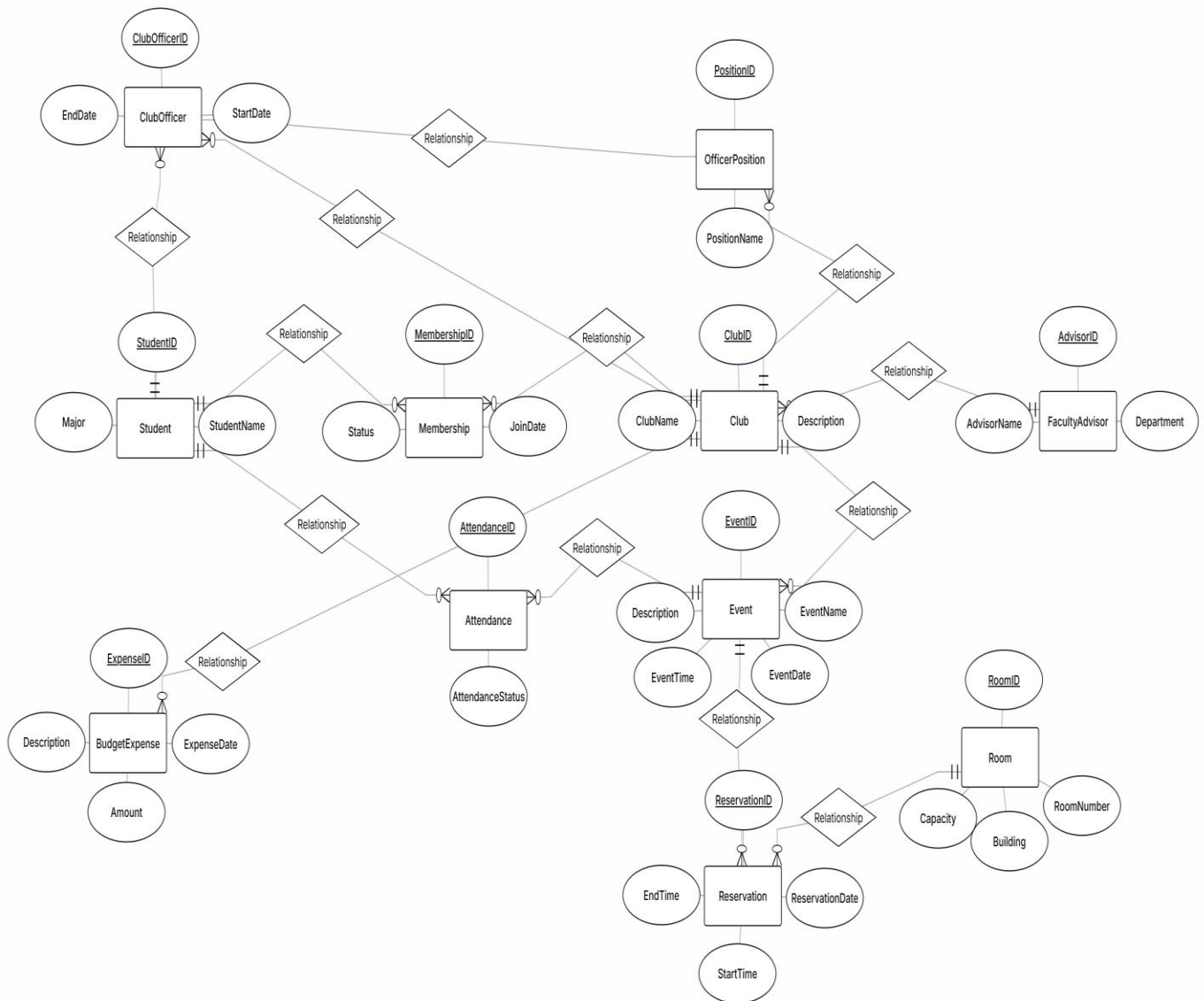
These relations are in BCNF because in each relation, the determinant of every non-trivial functional dependency is a superkey.

5) The decomposition is lossless, meaning that no information is lost when the decomposed relations are joined together. The original CourseSchedule information can be reconstructed using the common attributes between the relations, such as StudentID, CourseID, and TimeSlot. Therefore, there is no loss of information in this BCNF decomposition.

PART 5: Design Challenge

Task 5.1: Real-World Application

1)



2) Normalized Relational Schema

Student

Student(StudentID, StudentName, Major)

Primary Key: StudentID

Club

Club(ClubID, ClubName, Description, AdvisorID)

Primary Key: ClubID

Foreign Key: AdvisorID → FacultyAdvisor(AdvisorID)

FacultyAdvisor

FacultyAdvisor(AdvisorID, AdvisorName, Department)

Primary Key: AdvisorID

Event

Event(EventID, EventName, EventDate, EventTime, Description, ClubID)

Primary Key: EventID

Foreign Key: ClubID → Club(ClubID)

Room

Room(RoomID, RoomNumber, Building, Capacity)

Primary Key: RoomID

Reservation

Reservation(ReservationID, ReservationDate, StartTime, EndTime, EventID, RoomID)

Primary Key: ReservationID

Foreign Keys:

EventID → Event(EventID)

RoomID → Room(RoomID)

Membership

Membership(MembershipID, JoinDate, Status, StudentID, ClubID)

Primary Key: MembershipID

Foreign Keys:

StudentID → Student(StudentID)

ClubID → Club(ClubID)

Attendance

Attendance(AttendanceID, AttendanceStatus, StudentID, EventID)

Primary Key: AttendanceID

Foreign Keys:

StudentID → Student(StudentID)

EventID → Event(EventID)

OfficerPosition

OfficerPosition(PositionID, PositionName, ClubID)

Primary Key: PositionID

Foreign Key: ClubID → Club(ClubID)

ClubOfficer

ClubOfficer(ClubOfficerID, StartDate, EndDate, StudentID, ClubID, PositionID)

Primary Key: ClubOfficerID

Foreign Keys:

StudentID → Student(StudentID)

ClubID → Club(ClubID)

PositionID → OfficerPosition(PositionID)

BudgetExpense

BudgetExpense(ExpenseID, ExpenseDate, Amount, Description, ClubID)

Primary Key: ExpenseID

Foreign Key: ClubID → Club(ClubID)

3) Design Decision

One design decision was how to represent club membership. There were two valid options: using a direct many-to-many relationship between Student and Club, or creating a separate Membership entity.

I chose to create a separate Membership entity because it allows the database to store additional information about membership, such as JoinDate and Status. This makes the design more flexible and normalized.

4) Example Queries

1. Find all students who are members of the Computer Science Club.
2. List all club events scheduled for next week with their room reservations.
3. Find all students who hold officer positions in each club.